SISTEMAS OPERATIVOS MONOPUESTO (SMR 1.º)

Personalización del intérprete de comandos y del entorno de trabajo

Tipo Explicación | **Nivel** 3 · Avanzado | **Tema** 11 | **CM** CM4 (parcialmente) | **Duración** 10 h

Herramientas H2 (CMD/PowerShell), H4 (terminal Linux y Bash)

🧭 **Antes de empezar.** Llevas todo el curso usando el terminal tal como viene de fábrica. Esta guía trata de **configurarlo**: que se comporte como tú quieres, que la configuración sobreviva al reinicio y que puedas llevártela a otro equipo.

🧩 **Analogía.** El terminal por defecto es un coche de alquiler: arranca y va, pero el asiento no está a tu altura, los espejos están mal y no sabes dónde está nada. Configurarlo es ajustar el asiento, los espejos y las emisoras. Tarda diez minutos y luego conduces sin pensar. Y lo importante de esta unidad: **los ajustes se guardan en ficheros de texto**, así que puedes copiarlos a otro coche.

Índice

- Personalización del intérprete de comandos y del entorno de trabajo
  - Índice
  - 1. Cómo arranca un shell: qué fichero se lee y cuándo
    - Bash
    - Zsh
    - Regla práctica
  - 2. Personalizar Bash a mano
    - 2.1 El prompt: la variable `PS1`
    - 2.2 Alias
    - 2.3 Variables de entorno y `PATH`
    - 2.4 Historial
  - 3. Instalar una fuente Nerd Font
  - 4. Instalar y activar Zsh
  - 5. Powerlevel10k
    - 5.1 Instalación
    - 5.2 El asistente `p10k configure`
    - 5.3 Qué ha hecho el asistente

    - 5.4 El *instant prompt*
  - 6. Plugins de Zsh
  - 7. Herramientas modernas de terminal
    - Integración en `.zshrc`
  - 8. El fichero `.zshrc` completo
  - 9. Personalizar el emulador de terminal
  - 10. Lo mismo en PowerShell
    - 10.1 El fichero de perfil
    - 10.2 Directiva de ejecución
    - 10.3 Contenido del perfil
    - 10.4 Equivalencias
  - 11. Comprobar la persistencia y el aislamiento entre usuarios
    - 11.1 Persistencia
    - 11.2 Aislamiento entre usuarios
    - 11.3 Configuración para todos los usuarios
  - 12. Guardar la configuración: dotfiles
  - Problemas frecuentes
  - Ampliación
    - Starship
    - Oh My Posh en PowerShell
    - Otras propuestas

---

1. Cómo arranca un shell: qué fichero se lee y cuándo

Antes de tocar nada, hay que entender un detalle que confunde a todo el mundo.

Hay dos ejes independientes:

- **De inicio de sesión (*login*) o no.** Es de inicio de sesión cuando te identificas: al entrar por la consola de texto, o al conectarte por SSH. **No** lo es cuando abres una ventana de terminal dentro de un escritorio que ya tiene la sesión abierta.
- **Interactivo o no.** Es interactivo cuando hay una persona escribiendo. No lo es cuando el shell ejecuta un script.

Según la combinación, se leen unos ficheros u otros:

Bash

Situación	Ficheros que se leen, en orden
Inicio de sesión (consola, SSH)	`/etc/profile` → el primero que exista de `~/.bash_profile`, `~/.bash_login`, `~/.profile`
Interactivo sin inicio de sesión (terminal del escritorio)	`/etc/bash.bashrc` → `~/.bashrc`
Script	Ninguno de los anteriores

> ℹ️ Por eso la mayoría de los `~/.bash_profile` contienen esta línea: para que el de inicio de sesión cargue también el interactivo y no tengas que mantener dos ficheros.

```
[[ -f ~/.bashrc ]] && . ~/.bashrc
```

Zsh

Situación	Ficheros que se leen, en orden
Siempre	`/etc/zshenv` → `~/.zshenv`
Inicio de sesión	`~/.zprofile`
Interactivo	`~/.zshrc` ← **el que vas a usar tú**
Inicio de sesión, al salir	`~/.zlogout`

Regla práctica

Qué quieres poner	Dónde va
Alias, funciones, prompt, opciones de historial	`~/.zshrc` (o `~/.bashrc`)
Variables de entorno (`PATH`, `EDITOR`, `LANG`)	`~/.zprofile` (o `~/.bash_profile`)
Configuración para **todos** los usuarios	`/etc/profile.d/mi-config.sh`

Comprobación práctica. Averigua qué shell tienes y qué tipo de shell es:

```
echo $SHELL          # el shell asignado a tu usuario en /etc/passwd
echo $0              # el shell que se está ejecutando ahora mismo
shopt -q login_shell && echo "es login" || echo "no es login"
```

Prueba a poner un `echo` distinto al final de `~/.bashrc` y de `~/.bash_profile`, cierra sesión, vuelve a entrar por la consola y luego abre un terminal del escritorio. Verás cuál se ejecuta en cada caso. **Esto entra en el entregable.**

2. Personalizar Bash a mano

Antes de instalar nada, vamos a hacerlo «a pelo» en Bash. Powerlevel10k va a hacer esto mismo automáticamente, y si no lo has hecho una vez a mano no vas a entender qué hace.

2.1 El prompt: la variable `PS1`

El *prompt* es lo que se ve antes del cursor. Lo define la variable `PS1`. Pruébalo en caliente:

```
echo $PS1
PS1='hola > '
```

Vuelve a la normalidad abriendo un terminal nuevo (no lo has guardado en ningún sitio).

Códigos de escape útiles:

Código	Muestra
`\u`	Nombre de usuario
`\h`	Nombre del equipo (hasta el primer punto)
`\w`	Directorio actual, ruta completa
`\W`	Directorio actual, sólo el último nombre
`\$`	`#` si eres root, `$` si no
`\t`	Hora
`\n`	Salto de línea

Y los colores, con `\[\e[código m\]`:

Código	Color
`0;32`	Verde
`1;34`	Azul brillante
`1;31`	Rojo brillante
`0`	Reiniciar a color normal

Un prompt completo, coloreado y con la ruta en otra línea:

```
PS1='\[\e[1;32m\]\u@\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\n\$ '
```

> ⚠️ **Los `\[` y `\]` no son decoración.** Marcan qué caracteres son «invisibles» (no ocupan ancho). Si los olvidas, Bash calcula mal dónde está el cursor y al editar comandos largos la línea se pisa a sí misma. Es un error habitual y difícil de diagnosticar.

Para que sea permanente, añádelo al final de `~/.bashrc`:

```
echo "PS1='\[\e[1;32m\]\u@\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\n\$ '" >> ~/.bashrc
```

Y recarga sin cerrar el terminal:

```
source ~/.bashrc
```

> ℹ️ `source` (o su abreviatura `.`) ejecuta el fichero **en el shell actual**. Si lo ejecutaras con `bash ~/.bashrc` se abriría un shell nuevo, aplicaría los cambios ahí y al terminar se cerraría llevándoselos. Esta distinción vuelve a salir en la guía de scripting.

2.2 Alias

Un alias es un apodo para un comando.

```
alias ll='ls -lah'
alias ..='cd ..'
alias ...='cd ../..'
alias grep='grep --color=auto'
```

Ver los que tienes y borrar uno:

```
alias
unalias ll
```

Para ejecutar el comando **real** saltándote el alias, ponle una barra delante: `\grep`.

2.3 Variables de entorno y `PATH`

```
echo $PATH
which ls
```

`PATH` es la lista de carpetas donde el shell busca los programas, separadas por `:`. Cuando escribes `ls`, el shell recorre esas carpetas en orden hasta encontrarlo.

Para añadir tu propia carpeta de scripts:

```
mkdir -p ~/bin
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bash_profile
```

> ⚠️ `$HOME/bin:$PATH` y `$PATH:$HOME/bin` **no son lo mismo.** En el primer caso tu carpeta se mira **antes** que las del sistema, así que un script tuyo llamado `ls` se ejecutaría en lugar del `ls` real. Es una técnica útil... y también una vía de ataque conocida. Piensa qué orden quieres.

Otras variables útiles, en `~/.bash_profile`:

```
export EDITOR=nano
export VISUAL=nano
export LANG=es_ES.UTF-8
```

2.4 Historial

```
history           # ver el historial
!!                # repetir el último comando
!123              # repetir el comando número 123
!$                # el último argumento del comando anterior
Ctrl + R          # buscar hacia atrás en el historial
```

Configuración recomendada en `~/.bashrc`:

```
HISTSIZE=10000           # comandos en memoria
HISTFILESIZE=20000       # comandos guardados en el fichero
HISTCONTROL=ignoreboth   # no guarda duplicados ni líneas que empiezan por espacio
shopt -s histappend      # añade al fichero en lugar de sobrescribirlo
```

ℹ️ `HISTCONTROL=ignoreboth` tiene un uso práctico de seguridad: si escribes un comando con una contraseña dentro y lo empiezas **con un espacio**, no queda registrado en el historial.

3. Instalar una fuente Nerd Font

Hazlo antes de instalar Powerlevel10k. Sin esto verás cuadraditos y signos de interrogación en lugar de iconos, y pensarás que has hecho algo mal.

ℹ️ **Qué es una Nerd Font.** Una fuente monoespaciada normal a la que se le han añadido miles de iconos (ramas de Git, logos de sistemas, flechas, candados). Los prompts modernos usan esos iconos; si la fuente no los tiene, el terminal dibuja el carácter «no disponible».

En Arch:

```
sudo pacman -S ttf-meslo-nerd
fc-cache -fv
```

Comprueba que se han instalado:

```
fc-list | grep -i meslo
```

Después hay que **decirle al emulador de terminal que la use** (paso 9). Instalarla no basta.

ℹ️ En Ubuntu: `sudo apt install fonts-meslo-lgs-nf` o descarga manual desde https://www.nerdfonts.com y copia los `.ttf` a `~/.local/share/fonts/`, seguido de `fc-cache -fv`.

4. Instalar y activar Zsh

Zsh es otro intérprete de comandos, compatible con casi todo lo de Bash pero con mejor autocompletado, corrección de erratas y un sistema de temas mucho más potente.

```
sudo pacman -S zsh zsh-completions
```

Pruébalo sin cambiar nada todavía:

```
zsh
```

La primera vez sale el asistente `zsh-newuser-install`. Pulsa `q` para salir: vamos a escribir el `.zshrc` nosotros.

Sal con `exit` y ahora sí, cambia tu shell por defecto:

```
chsh -s /bin/zsh
```

> ℹ️ `chsh` cambia el shell registrado para tu usuario en `/etc/passwd`. Compruébalo:
>
> ```
> grep "^$USER:" /etc/passwd
> ```
>
> El último campo de esa línea es el shell. Es exactamente el fichero que analizasteis en la hoja de ejercicios «Explorar el sistema».

Cierra sesión y vuelve a entrar. `chsh` no cambia el shell de la sesión que ya está abierta.

> ⚠️ **No hagas `chsh -s /bin/zsh` para root.** Si algún día `zsh` se rompe o se desinstala, root se queda sin shell utilizable y recuperar el sistema es mucho más incómodo. Root se queda con Bash.

5. Powerlevel10k

Powerlevel10k (p10k) es un tema para Zsh. Muestra en el prompt información contextual: la rama de Git, si hay cambios sin guardar, si el último comando falló, cuánto ha tardado, si estás en una máquina remota, si eres root.

> ℹ️ **Nota honesta sobre su estado.** Powerlevel10k está desde 2024 en modo mantenimiento: su autor dejó el desarrollo activo. Sigue funcionando perfectamente y es el tema más usado, pero no esperes funciones nuevas. En la ampliación tienes **Starship**, la alternativa en desarrollo activo, que además funciona igual en Bash, Zsh y PowerShell.

5.1 Instalación

El método oficial, que funciona en cualquier distribución:

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git ~/.powerlevel10k
echo 'source ~/.powerlevel10k/powerlevel10k.zsh-theme' >> ~/.zshrc
```

> ℹ️ `--depth=1` descarga sólo la última versión, sin todo el histórico del proyecto. Baja de unos 30 MB a unos 3 MB.

Alternativa en Arch usando AUR (necesitas `paru`, paso 17 de la guía de instalación):

```
paru -S zsh-theme-powerlevel10k-git
```

Abre un terminal nuevo. Arrancará solo el asistente de configuración.

5.2 El asistente `p10k configure`

Si no arranca solo, o quieres volver a lanzarlo:

```
p10k configure
```

Te va preguntando. Las primeras preguntas son **de diagnóstico de la fuente**: te enseña un rombo, un candado, un icono... y te pregunta si los ves bien. Si respondes «no», da por hecho que no tienes Nerd Font y usa un modo degradado.

> ℹ️ Si ves cuadraditos, **no sigas respondiendo que no**. Sal con `Ctrl` + `C`, vuelve al paso 3, configura la fuente en el terminal (paso 9) y relanza `p10k configure`.

Respuestas recomendadas para clase:

Pregunta	Recomendación
Prompt Style	**Rainbow** (el más informativo) o Lean (el más limpio)
Character Set	**Unicode**
Show current time?	**24-hour format**
Prompt Separators / Heads / Tails	cualquiera, es estético
Prompt Height	**Two lines** (el comando empieza siempre en la misma columna)
Prompt Connection	Dotted
Prompt Frame	Left

Pregunta	Recomendación
Prompt Spacing	**Sparse** (deja una línea en blanco entre comandos; se lee mucho mejor)
Icons	**Many icons**
Prompt Flow	Concise
Enable Transient Prompt?	**Yes** (los comandos anteriores se simplifican; el historial queda limpio)
Instant Prompt Mode	**Verbose**

El asistente guarda todo en `~/.p10k.zsh` y añade la línea que lo carga a tu `~/.zshrc`.

5.3 Qué ha hecho el asistente

Ábrelo y mira:

```
less ~/.p10k.zsh
grep -n "POWERLEVEL9K_LEFT_PROMPT_ELEMENTS" -A 20 ~/.p10k.zsh
```

Verás una lista de **segmentos**: `dir`, `vcs`, `status`, `command_execution_time`... Cada segmento es un trozo del prompt. Puedes quitar o reordenar los que quieras y recargar con `source ~/.zshrc`.

> ❗ **Esto es lo que en el paso 2 hacías a mano con `PS1`.** p10k no hace magia: genera un `PS1` muy largo por ti. Compruébalo con `echo $PS1` estando en zsh con p10k activo.

5.4 El *instant prompt*

Al principio del `.zshrc` el asistente añade un bloque como éste:

```
if [[ -r "${XDG_CACHE_HOME:-$HOME/.cache}/p10k-instant-prompt-${(%):-%n}.zsh" ]]; then
  source "${XDG_CACHE_HOME:-$HOME/.cache}/p10k-instant-prompt-${(%):-%n}.zsh"
fi
```

Dibuja el prompt inmediatamente con una versión guardada en caché mientras el resto de la configuración termina de cargarse. Por eso el terminal parece abrirse al instante.

> ⚠️ **Ese bloque tiene que quedarse arriba del todo** y no debe haber nada antes que escriba en pantalla. Si añades un `echo` o un `fastfetch` por encima, verás un aviso de p10k en cada arranque. La solución es poner esas cosas **al final** del `.zshrc`.

6. Plugins de Zsh

Dos plugins que cambian de verdad cómo se usa el terminal:

```
sudo pacman -S zsh-autosuggestions zsh-syntax-highlighting
```

Plugin	Qué hace
`zsh-autosuggestions`	Mientras escribes, sugiere en gris el comando completo del historial. `→` lo acepta
`zsh-syntax-highlighting`	Colorea el comando mientras lo escribes: verde si existe, rojo si no. Detectas la errata **antes** de pulsar Intro

Se cargan añadiendo estas líneas a `~/.zshrc`:

```
source /usr/share/zsh/plugins/zsh-autosuggestions/zsh-autosuggestions.zsh
source /usr/share/zsh/plugins/zsh-syntax-highlighting/zsh-syntax-highlighting.zsh
```

> ⚠️ `zsh-syntax-highlighting` **tiene que ser lo último que se cargue** de los dos, o dejará de colorear parte de lo que escribes. Está documentado en su propio README, y es el tipo de detalle que sólo se aprende leyendo la documentación.

Sobre Oh My Zsh. Es un *framework* que instala decenas de plugins de golpe. Es cómodo, pero para esta unidad **no lo usamos**: el objetivo es que sepas qué hace cada línea de tu configuración, y Oh My Zsh esconde justo eso. Con dos plugins y p10k tienes lo mismo y sabes por qué funciona.

7. Herramientas modernas de terminal

Sustitutos actuales de comandos clásicos. La gracia de la mayoría es que se instalan y se enchufan como alias, así que sigues escribiendo lo de siempre.

```
sudo pacman -S eza bat fd ripgrep fzf zoxide btop duf ncdu tealdeer fastfetch
```

Herramienta	Sustituye a	Qué aporta
`eza`	`ls`	Colores por tipo, iconos, árbol, estado de Git
`bat`	`cat`	Numera líneas, resalta sintaxis, pagina
`fd`	`find`	Sintaxis mucho más simple y respeta `.gitignore`

Herramienta	Sustituye a	Qué aporta
`ripgrep` (`rg`)	`grep -r`	Búsqueda recursiva muy rápida
`fzf`	—	Buscador interactivo difuso; mejora `Ctrl` + `R`
`zoxide`	`cd`	Aprende tus carpetas: `z proyectos` te lleva sin la ruta completa
`btop`	`top` / `htop`	Monitor de recursos gráfico (útil para diagnosticar lentitud)
`duf`	`df -h`	Espacio en disco en tabla legible
`ncdu`	`du`	Explorador interactivo de qué ocupa espacio
`tealdeer` (`tldr`)	`man`	Ejemplos prácticos en vez del manual completo
`fastfetch`	`neofetch`	Información del sistema. `neofetch` está descontinuado desde 2024

Primer uso de `tealdeer` (hay que descargar su base de datos):

```
tldr --update
tldr tar
```

> ⚠️ **Aviso importante.** Estas herramientas son cómodas, pero **en un examen o en un servidor ajeno no van a estar**. `ls`, `cat`, `grep`, `find` y `df` están en todos los sistemas Unix del mundo. Aprende primero los originales; usa los sustitutos por comodidad, no como muleta. La prueba práctica se hace con los comandos estándar.

Integración en `.zshrc`

`fzf` y `zoxide` no funcionan solos: hay que engancharlos al shell.

```
source /usr/share/fzf/key-bindings.zsh
source /usr/share/fzf/completion.zsh
eval "$(zoxide init zsh)"
```

8. El fichero `.zshrc` completo

Éste es el resultado de todo lo anterior. **Es la base del entregable**, pero no lo copies tal cual: tienes que entender cada bloque, quitar lo que no uses y añadir tus propios alias con tus propios comentarios.

```
# ============================================================
#  ~/.zshrc — configuración de Zsh
#  Autor: <tu nombre>   ·  SMR 1.º — Sistemas Operativos Monopuesto
#  Se lee en cada shell INTERACTIVO (ver apartado 1 de la guía)
# ============================================================

# --- Instant prompt de Powerlevel10k -------------------------
# Debe ir SIEMPRE lo primero. Nada que escriba en pantalla puede ir antes.
if [[ -r "${XDG_CACHE_HOME:-$HOME/.cache}/p10k-instant-prompt-${(%):-%n}.zsh" ]]; then
  source "${XDG_CACHE_HOME:-$HOME/.cache}/p10k-instant-prompt-${(%):-%n}.zsh"
fi

# --- Tema ----------------------------------------------------
source ~/.powerlevel10k/powerlevel10k.zsh-theme
[[ ! -f ~/.p10k.zsh ]] || source ~/.p10k.zsh   # ajustes de 'p10k configure'

# --- Historial -----------------------------------------------
HISTFILE=~/.zsh_history
HISTSIZE=10000                  # comandos en memoria
SAVEHIST=20000                  # comandos guardados en disco
setopt HIST_IGNORE_ALL_DUPS     # no guarda duplicados
setopt HIST_IGNORE_SPACE        # lo que empieza por espacio no se guarda
setopt SHARE_HISTORY            # historial compartido entre terminales abiertos
setopt INC_APPEND_HISTORY       # guarda al momento, no al cerrar

# --- Comportamiento del shell --------------------------------
setopt AUTO_CD                  # escribir 'Documentos' equivale a 'cd Documentos'
setopt CORRECT                  # sugiere corrección de erratas en comandos
setopt EXTENDED_GLOB            # comodines avanzados
setopt NO_BEEP                  # sin pitido

# --- Autocompletado ------------------------------------------
autoload -Uz compinit && compinit
zstyle ':completion:*' menu select                      # menú navegable con flechas
zstyle ':completion:*' matcher-list 'm:{a-zA-Z}={A-Za-z}' # ignora mayúsculas/minúsculas

# --- Variables de entorno ------------------------------------
export EDITOR=nano
export VISUAL=nano
export PAGER=less
export PATH="$HOME/bin:$PATH"   # mis scripts, antes que los del sistema

# --- Alias: navegación ---------------------------------------
alias ..='cd ..'
alias ...='cd ../..'
alias ....='cd ../../..'

# --- Alias: listados (eza sustituye a ls) --------------------
alias ls='eza --icons --group-directories-first'
alias ll='eza -l --icons --group-directories-first --git'
alias la='eza -la --icons --group-directories-first --git'
alias lt='eza --tree --level=2 --icons'

# --- Alias: ficheros y búsqueda ------------------------------
alias cat='bat --paging=never'
alias grep='grep --color=auto'
alias df='duf'
```

```
# --- Alias: seguridad ------------------------------------
# -i pide confirmación antes de sobrescribir o borrar.
# Es una red de seguridad, NO una excusa para no leer lo que escribes.
alias cp='cp -i'
alias mv='mv -i'
alias rm='rm -i'

# --- Alias: sistema --------------------------------------
alias actualizar='sudo pacman -Syu'
alias buscar='pacman -Ss'
alias servicios='systemctl list-units --type=service --state=running'
alias puertos='ss -tulpn'
alias micro-ip='ip -br a'

# --- Funciones -------------------------------------------
# Crear un directorio y entrar en él de una vez
mkcd() { mkdir -p "$1" && cd "$1"; }

# Copia de seguridad rápida de un fichero, con fecha
respaldo() { cp -v "$1" "$1.$(date +%Y%m%d-%H%M).bak"; }

# Extraer cualquier archivo comprimido sin recordar las opciones
extraer() {
  case "$1" in
    *.tar.gz|*.tgz)  tar -xzf "$1" ;;
    *.tar.bz2)       tar -xjf "$1" ;;
    *.tar)           tar -xf  "$1" ;;
    *.zip)           unzip    "$1" ;;
    *.gz)            gunzip   "$1" ;;
    *) echo "Formato no reconocido: $1" ;;
  esac
}

# --- Integraciones ---------------------------------------
source /usr/share/fzf/key-bindings.zsh     # Ctrl+R y Ctrl+T interactivos
source /usr/share/fzf/completion.zsh
eval "$(zoxide init zsh)"                  # comando 'z'

# --- Plugins ---------------------------------------------
source /usr/share/zsh/plugins/zsh-autosuggestions/zsh-autosuggestions.zsh
# syntax-highlighting SIEMPRE el último de los plugins
source /usr/share/zsh/plugins/zsh-syntax-highlighting/zsh-syntax-highlighting.zsh

# --- Al abrir el terminal --------------------------------
# Va al final para no interferir con el instant prompt de p10k
fastfetch
```

Aplica los cambios:

```
source ~/.zshrc
```

> 💡 **Consejo antes de tocar el fichero:** haz una copia. `cp ~/.zshrc ~/.zshrc.bak`. Un error de sintaxis aquí puede dejarte con un terminal que no arranca bien, y tendrás que arreglarlo desde otra consola virtual (`Ctrl` + `Alt` + `F2`).

9. Personalizar el emulador de terminal

La fuente, los colores y la transparencia **no** los controla el shell: los controla el programa que dibuja la ventana. Es una distinción que cuesta ver al principio.

> 🧩 **Analogía.** El shell es el locutor de radio; el emulador de terminal es el aparato de radio. Las palabras las pone el locutor, pero el volumen, los graves y el color de la carcasa son del aparato. Si los iconos se ven mal, el problema está en el aparato, no en el locutor.

En XFCE Terminal (el de la MV de Arch): **Editar** → **Preferencias** → **Apariencia**

- Fuente: **MesloLGS Nerd Font Regular**, tamaño 11 o 12
- Desmarcar «Usar la fuente del sistema»
- **Colores**: elige un esquema con buen contraste (Solarized Dark, Tango, Nord)
- Desplazamiento: **10000 líneas**, para poder subir a ver la salida de un comando largo

En GNOME Terminal: Preferencias → Perfil → Texto → Fuente personalizada.

En Windows Terminal (para el paso 10): Configuración → tu perfil → Apariencia → Tipo de letra.

Comprueba que la fuente es correcta:

```
echo -e "\ue0b0 \uf303 \ue725 \uf09b \uf015"
```

Debes ver: una flecha maciza, el logo de Arch, una rama de Git, el logo de GitHub y una casita. Si ves cuadraditos, la fuente del terminal no está bien seleccionada.

10. Lo mismo en PowerShell

Esta guía pide **los dos intérpretes**. Esto se hace en la máquina virtual de Windows.

10.1 El fichero de perfil

El equivalente a `.zshrc` en PowerShell es el **perfil**:

```
$PROFILE                        # ruta del fichero de perfil
Test-Path $PROFILE              # ¿existe?
New-Item -ItemType File -Path $PROFILE -Force   # crearlo
notepad $PROFILE                # editarlo
```

Suele estar en `C:\Users\<usuario>\Documents\PowerShell\Microsoft.PowerShell_profile.ps1`.

> ℹ️ Igual que en Linux, es **por usuario**. El perfil de todos los usuarios está en `$PSHOME\Profile.ps1`. La estructura del problema es idéntica a la de `~/.zshrc` frente a `/etc/profile.d/`.

10.2 Directiva de ejecución

Windows no ejecuta scripts de PowerShell por defecto. Para que se cargue tu perfil:

```
Get-ExecutionPolicy -List
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

`RemoteSigned` permite ejecutar scripts locales y exige firma a los descargados de internet. **No pongas** `Unrestricted`: es exactamente el ajuste que buscan los ataques por script.

10.3 Contenido del perfil

```
# ---- Perfil de PowerShell ----

# Alias
Set-Alias ll Get-ChildItem
Set-Alias grep Select-String
Set-Alias which Get-Command

# Funciones
function .. { Set-Location .. }
function ... { Set-Location ..\.. }
function mkcd { param($ruta) New-Item -ItemType Directory -Path $ruta -Force; Set-Location $ruta }

# Prompt personalizado (equivalente a PS1)
function prompt {
    $ruta = (Get-Location).Path.Replace($HOME, "~")
    Write-Host "$env:USERNAME@$env:COMPUTERNAME" -NoNewline -ForegroundColor Green
    Write-Host ":" -NoNewline
    Write-Host $ruta -ForegroundColor Blue
    return "PS> "
}

# Historial de sesiones anteriores (PSReadLine)
Set-PSReadLineOption -PredictionSource History
Set-PSReadLineOption -HistorySearchCursorMovesToEnd
Set-PSReadLineKeyHandler -Key UpArrow -Function HistorySearchBackward
```

Recarga el perfil sin cerrar la ventana (el punto y el espacio son parte del comando):

```
. $PROFILE
```

10.4 Equivalencias

Concepto	Zsh / Bash	PowerShell
Fichero de configuración	`~/.zshrc`	`$PROFILE`
Recargar configuración	`source ~/.zshrc`	`. $PROFILE`
Alias	`alias ll='ls -la'`	`Set-Alias ll Get-ChildItem`
Función	`mkcd() { ... }`	`function mkcd { ... }`
Prompt	variable `PS1`	función `prompt`
Variable de entorno	`export EDITOR=nano`	`$env:EDITOR = "notepad"`
Ver el historial	`history`	`Get-History`
Buscar en el historial	`Ctrl` + `R`	`Ctrl` + `R` (PSReadLine)
Permitir scripts	`chmod +x`	`Set-ExecutionPolicy`

> ℹ️ Si quieres un prompt tipo Powerlevel10k en PowerShell, el equivalente se llama **Oh My Posh** (`winget install JanDeDobbeleer.OhMyPosh`). Va en la ampliación.

11. Comprobar la persistencia y el aislamiento entre usuarios

Esto es lo que se califica. Instalar es la parte fácil.

11.1 Persistencia

```
sudo reboot
```

Al volver a entrar, comprueba que sigue estando todo: el prompt, los alias (`alias | head`), las funciones (`type mkcd`), y que `fastfetch` se ejecuta al abrir el terminal.

11.2 Aislamiento entre usuarios

Crea un segundo usuario y comprueba que **no** ha heredado nada:

```
sudo useradd -m -s /bin/bash prueba
sudo passwd prueba
su - prueba
```

Ya como `prueba`:

```
echo $SHELL      # /bin/bash, no zsh
alias            # no están tus alias
ls ~             # su home está vacío: no hay .zshrc
exit
```

> ❗ **Conclusión que debes escribir en el entregable:** la configuración del shell vive en el directorio personal de cada usuario, así que es **por usuario**. Cambiarla no afecta a los demás. Esto conecta directamente con usuarios y permisos: los ficheros de configuración están en `/home/alumno/` y sus permisos impiden que otro usuario los lea o los modifique. Compruébalo:
>
> ```
> ls -la /home/alumno/.zshrc
> ```

11.3 Configuración para todos los usuarios

Si quisieras un alias para **todo** el sistema:

```
sudo nano /etc/profile.d/alias-comunes.sh
```

```
alias ll='ls -lah'
```

Cualquier usuario nuevo lo tendrá. Y lo que se copie en `/etc/skel/` se copia al `home` de cada usuario que se cree a partir de ese momento — la misma `/etc/skel` que aparece al crear usuarios.

12. Guardar la configuración: dotfiles

Los ficheros de configuración empiezan por punto, por eso se llaman *dotfiles*. Como son texto, se pueden versionar con Git y llevar a cualquier equipo.

```
mkdir -p ~/dotfiles
cp ~/.zshrc ~/.p10k.zsh ~/dotfiles/
cd ~/dotfiles
git init
git add .
git commit -m "Configuración de zsh y powerlevel10k"
```

Para restaurar en otro equipo bastaría con clonar el repositorio y copiar los ficheros al `home`.

> ℹ️ Esto es una copia de seguridad en miniatura, y es un buen aperitivo de la guía de scripting: un script que copie automáticamente tus dotfiles con la fecha en el nombre es prácticamente el ejercicio de copia de seguridad con rotación.

Problemas frecuentes

Síntoma	Causa	Solución
Cuadraditos o `?` en el prompt	El terminal no usa una Nerd Font	Paso 3 y paso 9. Instalar la fuente **y** seleccionarla en el emulador
«Console output during zsh initialization detected»	Algo escribe en pantalla antes del *instant prompt*	Mover ese comando (p. ej. `fastfetch`) al final del `.zshrc`
Tras `chsh` sigue saliendo Bash	La sesión abierta no cambia de shell	Cerrar sesión y volver a entrar. Verificar con `grep "^$USER:" /etc/passwd`
Los alias funcionan y al reiniciar desaparecen	Se escribieron en el terminal, no en `~/.zshrc`	Añadirlos al fichero y hacer `source`
El coloreado de sintaxis deja de funcionar a medias	`zsh-syntax-highlighting` no es el último `source`	Moverlo al final de la sección de plugins
`command not found: eza`	El paquete no está instalado, o el alias apunta a algo inexistente	`sudo pacman -S eza` o quitar el alias
El terminal no arranca tras editar `.zshrc`	Error de sintaxis	`Ctrl` + `Alt` + `F2`, entrar en consola, `nano ~/.zshrc` o `cp ~/.zshrc.bak ~/.zshrc`
El perfil de PowerShell no se carga	Directiva de ejecución restrictiva	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Ampliación

Starship

Alternativa a Powerlevel10k en desarrollo activo, escrita en Rust, que funciona igual en **Bash, Zsh, Fish y PowerShell** — muy útil justo en esta unidad, porque configuras un solo prompt para los dos sistemas.

```
sudo pacman -S starship
echo 'eval "$(starship init zsh)"' >> ~/.zshrc
```

Se configura con un único fichero `~/.config/starship.toml`:

```
add_newline = true

[directory]
truncation_length = 3
style = "bold blue"

[git_branch]
symbol = " "

[cmd_duration]
min_time = 500
format = "took [$duration](bold yellow) "
```

> ℹ️ Para comparar de verdad: mide con `time zsh -i -c exit` cuánto tarda en arrancar el shell con p10k y con Starship. Escribe qué diferencias encuentras en velocidad, en información mostrada y en facilidad de configuración.

Oh My Posh en PowerShell

```
winget install JanDeDobbeleer.OhMyPosh
oh-my-posh init pwsh | Invoke-Expression
```

Otras propuestas

- Configurar `tmux` para tener varias sesiones en un solo terminal y que sobrevivan al cierre.
- Escribir un script que instale tu configuración completa en una máquina recién instalada (paquetes + dotfiles) en un solo comando. Es el puente directo con la guía de scripting.
- Comparar el consumo de memoria de `zsh` con y sin plugins usando `btop`.